

INHERITANCE

Aim:

To understand and implement inheritance concepts in Java.

PRE LAB EXERCISE

QUESTIONS

1. What is inheritance?

Inheritance is an OOP concept where one class (**child/subclass**) **acquires the properties and methods** of another class (**parent/superclass**). It helps in building new classes using existing ones.

2. What is code reusability?

Code reusability means **using the same code multiple times** instead of writing it again.

In Java, inheritance allows reusing:

- Variables
- Methods
- Logic of an existing class

3. What is the use of extends keyword?

The **extends keyword** is used to:

- Create a **subclass**
- Inherit another class's properties and methods

Syntax:

```
class ChildClass extends ParentClass {  
  
}
```

IN LAB EXERCISE

Objective:

To implement all types of inheritance.

PROGRAMS:

Student Result System (Single Inheritance)

Question:

A school wants to store student details and calculate marks. Create a base class Student and a derived class Result.

Code:

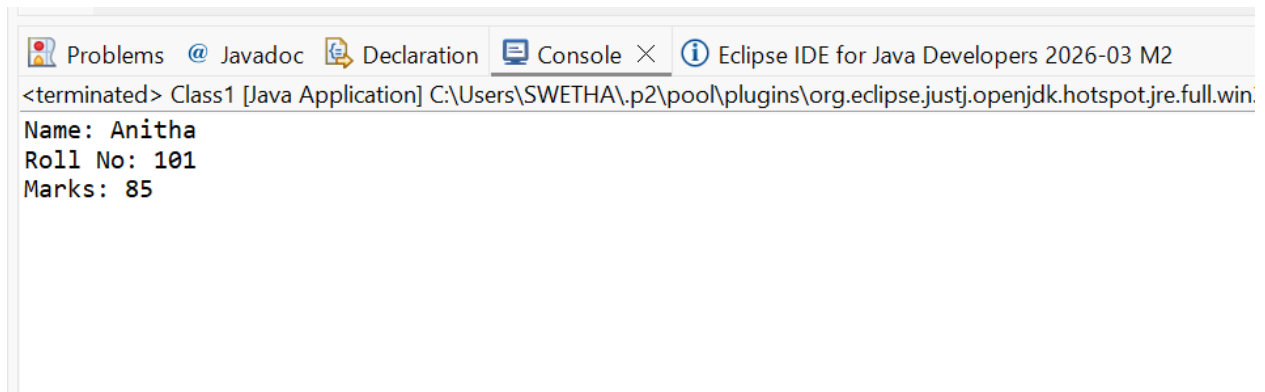
```
class Student {  
    String name;  
    int rollNo;  
  
    void getDetails() {  
        name = "Anitha";  
        rollNo = 101;  
    }  
}  
  
class Result extends Student {  
    int marks = 85;  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Marks: " + marks);  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Result r = new Result();
        r.getDetails();
        r.display();
    }
}

```

Output:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the results of the program execution: 'Name: Anitha', 'Roll No: 101', and 'Marks: 85'. The IDE title bar indicates 'Eclipse IDE for Java Developers 2026-03 M2'.

```

<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win.
Name: Anitha
Roll No: 101
Marks: 85

```

2. Bank Account System (Hierarchical Inheritance)

Question:

A bank has Savings and Current accounts. Both inherit from a common Account class.

Code:

```

class Account {
    void showAccountType() {
        System.out.println("Bank Account");
    }
}

class SavingsAccount extends Account {
    void interest() {

```

```
        System.out.println("Savings Account gives interest");
    }
}

class CurrentAccount extends Account {
    void overdraft() {
        System.out.println("Current Account supports overdraft");
    }
}

public class Main {
    public static void main(String[] args) {
        SavingsAccount s = new SavingsAccount();
        CurrentAccount c = new CurrentAccount();

        s.showAccountType();
        s.interest();

        c.showAccountType();
        c.overdraft();
    }
}
```

Output:

```
Problems JavaDoc Declaration Console Eclipse IDE for Java Developers 2020-03-12
<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.
Bank Account
Savings Account gives interest|
Bank Account
Current Account supports overdraft
```

3. Vehicle System (Multilevel Inheritance)

Question:

A company classifies vehicles as Vehicle → Car → ElectricCar.

Code:

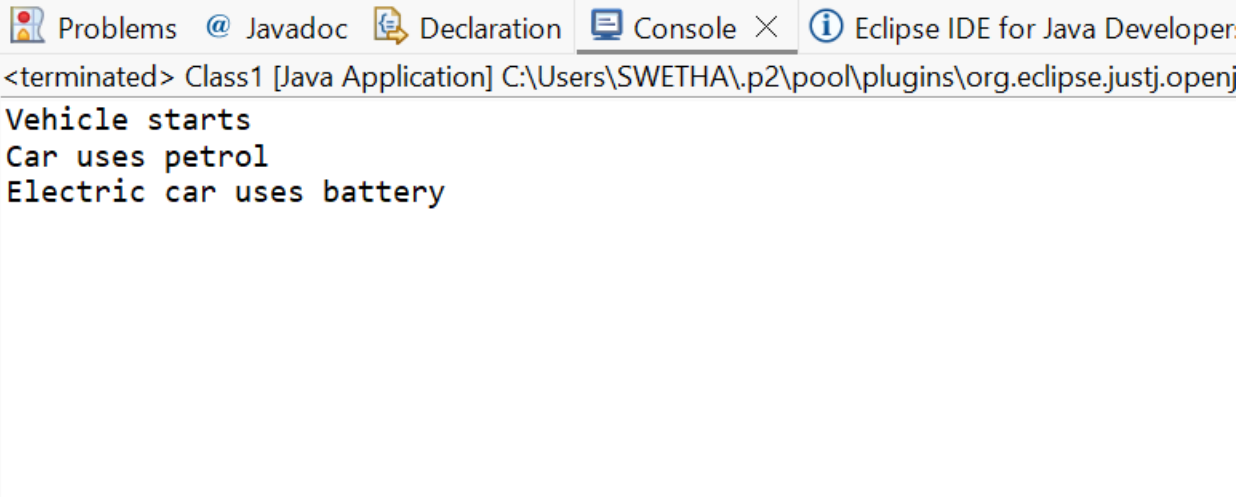
```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    void fuelType() {
        System.out.println("Car uses petrol");
    }
}

class ElectricCar extends Car {
    void battery() {
        System.out.println("Electric car uses battery");
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        ElectricCar e = new ElectricCar();  
        e.start();  
        e.fuelType();  
        e.battery();  
    }  
}
```

Output:

A screenshot of the Eclipse IDE's console window. The window has a title bar with icons for Problems, Javadoc, Declaration, Console, and Eclipse IDE for Java Developers. The console text shows the output of a Java application: "<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.openj", "Vehicle starts", "Car uses petrol", and "Electric car uses battery".

```
<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.openj  
Vehicle starts  
Car uses petrol  
Electric car uses battery
```

POST LAB EXERCISE

1. Why Java does not support multiple inheritance using classes and how it is implemented?

Java does **not support multiple inheritance with classes** to avoid **ambiguity and complexity**, mainly the **Diamond Problem**.

Diamond Problem:

If two parent classes have the same method, Java cannot decide **which method to inherit**.

How Java implements it instead:

Java supports multiple inheritance using **interfaces**, where:

- Methods are **abstract by default**
- Conflicts are resolved by **overriding methods**

2. What is the role of the super keyword? Give examples.

The **super keyword** is used to:

1. Access **parent class variables**
2. Call **parent class methods**
3. Call **parent class constructors**

Example:

```
class Parent {  
  
    int x = 10;  
  
}  
  
class Child extends Parent {  
  
    int x = 20;  
  
    void display() {  
  
        System.out.println(super.x); // prints 10  
  
    }  
  
}
```

3.Can a child class access private members of the parent class? Why?

No, a child class **cannot directly access private members** of the parent class.

Why?

- private members are **restricted to the same class**
- This ensures **data hiding and security**

4.Explain why hybrid inheritance is not supported in Java.

Hybrid inheritance is a combination of:

- Multiple inheritance
- Multilevel inheritance

Java does **not support hybrid inheritance using classes** because:

- It can cause **ambiguity**
- Leads to **method conflicts**
- Increases **complexity**

Java supports hybrid inheritance **using interfaces**, avoiding conflicts.

Result:

Thus the different types of inheritance were implemented and executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		